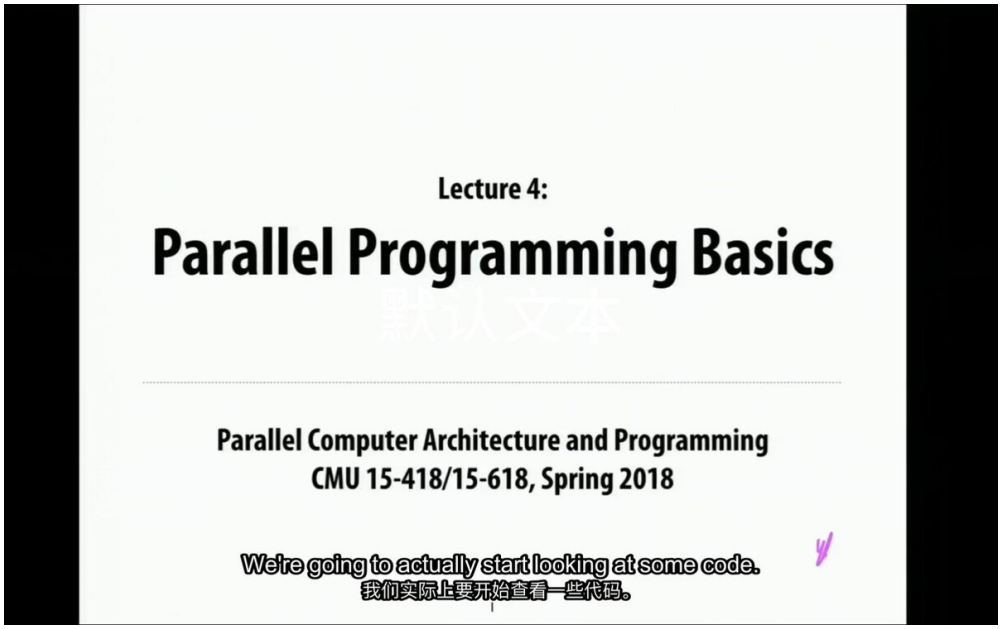
课程笔记

并行编程基础：分解、分配、编排与映射

CMU 15-418/618 并行计算机架构与编程 · Lecture 4（Spring 2018）

lecture-to-notes

2026 年 10 月 28 日



视频作者/频道：CMU 15-418/618（B 站搬运：alexphil）
发布日期：2018 年春季学期
视频时长：1 小时 19 分 38 秒
视频链接：`https://www.bilibili.com/video/BV1vu4y1R7rS/`

目录

1 从两个例子开始：规则结构与不规则结构 3

1.1 海洋模拟：规则的稠密网格 . . . 3
1.2 n-body 模拟：稀疏且不规则 . . . 4
1.3 四叉树与近似 . . . 4
1.4 本章小结 . . . 5

2 并行化的四个步骤 5

2.1 总览 . . . 5
2.2 分解 . . . 6
2.3 任务粒度的选择 . . . 7
2.4 本章小结 . . . 8
2.5 分配的目标 . . . 8
2.6 编排 . . . 8
2.7 映射 . . . 9
2.8 本章小结 . . . 9

3 Amdahl 定律：串行部分设下的天花板 10

3.1 一个常见错误 . . . 10
3.2 图像处理的例子 . . . 10
3.3 只并行第一步的后果 . . . 10
3.4 把第二步也并行：部分和 . . . 11
3.5 定律的一般形式 . . . 12
3.6 本章小结 . . . 13

4 分配的两种策略：静态与动态 14

4.1 静态分配与它的问题 . . . 14
4.2 一个具体写法与它暴露的两个问题 . . . 14
4.3 本章小结 . . . 15

5 动态分配：用队列换均衡 15

5.1 工作队列的做法 . . . 15
5.2 本章小结 . . . 16

6 编排：用红黑格序消除数据竞争 16

6.1 网格求解器的串行版本 . . . 16
6.2 依赖在哪里 . . . 17
6.3 双缓冲与它为什么被拒绝 . . . 18
6.4 红黑格序 . . . 18
6.5 阶段之间需要同步 . . . 19
6.6 屏障的代价与更精确的依赖 . . . 20
6.7 用多份副本减少屏障 . . . 21
6.8 本章小结 . . . 22

7 消息传递：同一个求解器的另一种编排 **22**
7.1 通信只能靠消息 23
7.2 死锁与它的解法 23
7.3 本章小结 24

8 总结与延伸 **24**

1　从两个例子开始：规则结构与不规则结构

在讨论方法之前，讲者先给出两个参照对象。它们会贯穿全课，也是后面讲性能优化的对象。

1.1　海洋模拟：规则的稠密网格

假设你是海洋学家，要模拟海洋的物理过程。海洋是三维的，但因为深度远小于水平范围，它像一个非常薄的立体。又因为压力与温度随深度变化，实际表示方式是一叠位于不同深度的二维平面。

每一层就是一个二维数组。空间与时间本来都是连续的，软件里必须离散化：把二维空间划成网格点，数组元素代表该坐标处的状态；再把时间切成步，逐个时间步推进。

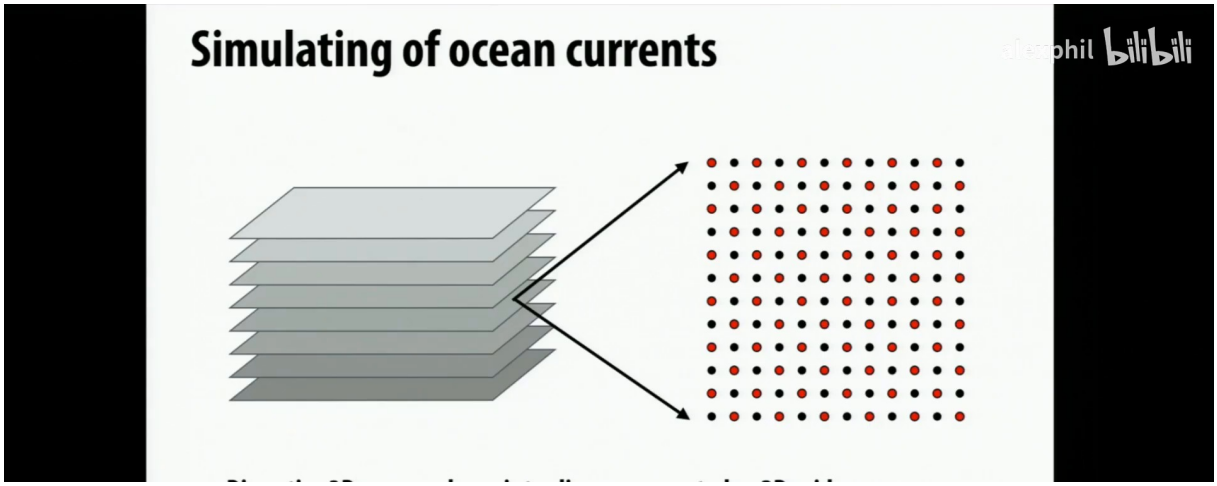


图 1: 海洋流模拟：把三维场按深度切成二维层逐层计算[1]

每一步要做的是求解一组偏微分方程，更新所有网格元素。讲者指出这正是程序花掉大部分时间的地方，也是后面并行化的重点。这个例子的显著特征是结构规整：一个稠密的二维数组，按时间步反复更新。

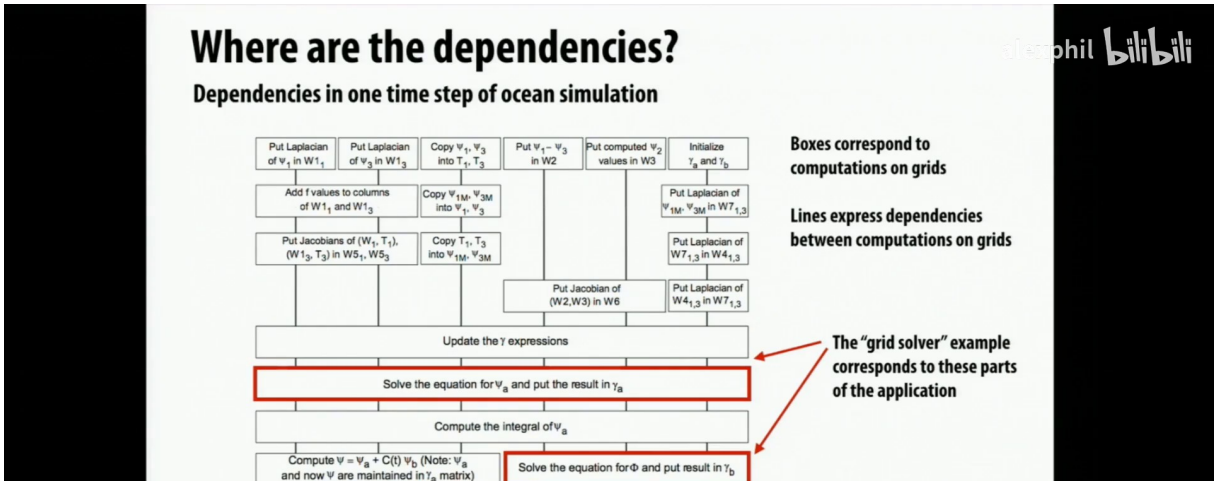


图 2: 单个时间步内的数据依赖[2]

[1] 视频画面时间区间：00:01:30–00:01:45。若为裁剪图，时间区间仍对应原视频画面。

[2] 视频画面时间区间：00:03:30–00:03:45。若为裁剪图，时间区间仍对应原视频画面。

1.2 n-body 模拟：稀疏且不规则

第二个例子模拟宇宙中的恒星。同样按时间步推进，但计算的是相互之间的引力：每个星体对其他星体施力，从而移动。

与海洋的例子不同，星体的空间分布极不均匀——有的地方挤成一团，有的地方很空。因此不能用稠密数组表示，否则大部分空间都是无意义的。这是稀疏数据结构，可以理解成一张图。

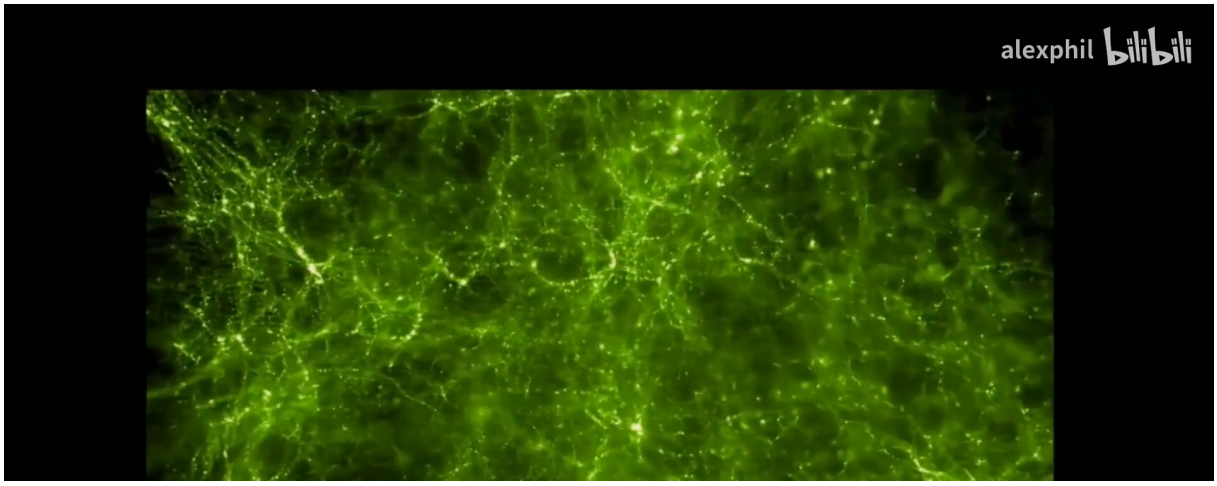


图 3: 星系演化模拟与 Barnes-Hut 算法[3]

朴素算法不可行：每个星体与每个其他星体都算一次，规模一大就完全不可扩展。

1.3 四叉树与近似

出路是近似。对某个待更新的星体，足够近的星体逐个计算；足够远的一群星体，则把它们合并成一个近似的质量，当作那里只有一颗大星。

近似算法把正确性换成了可控误差

把远处的星群合并为一个近似质量，意味着结果不再等于精确的万有引力求和。误差量级由半径参数控制：半径越大，聚合越多，算得越快，误差也越大。因此这类算法的参数选择本身就是精度与速度的权衡，而不是实现细节。

这样省下大量计算。分界由一个半径参数决定。

[3] 视频画面时间区间：00:05:30–00:05:45。若为裁剪图，时间区间仍对应原视频画面。

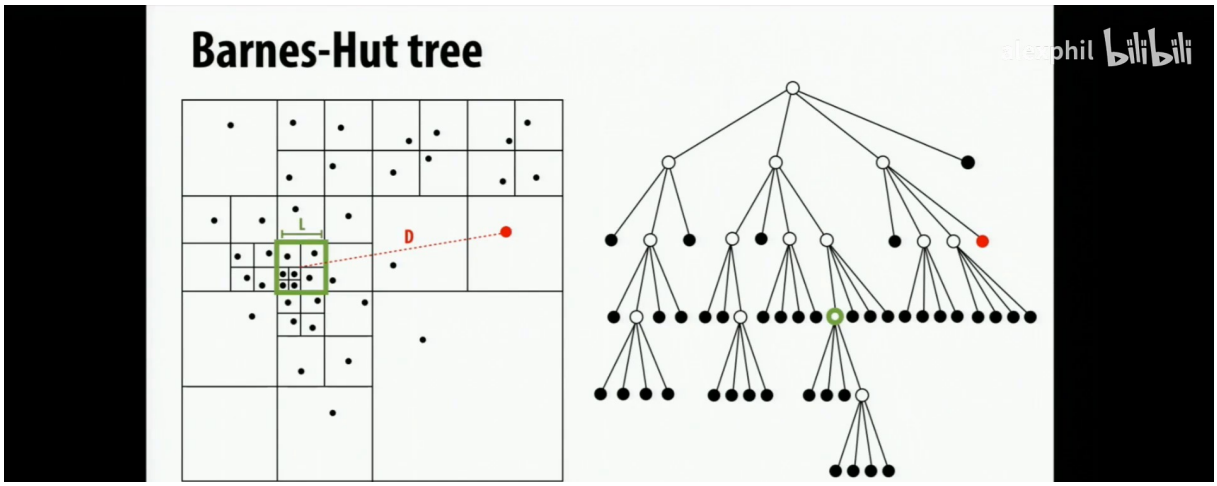


图 4: Barnes-Hut 四叉树：递归细分空间，邻近查询只需上溯几层[4]

要快速判断哪些星体近、哪些远，需要一个能表达空间位置的数据结构，这就是四叉树（三维时是八叉树）。讲者展示的是二维版本，因为便于画在幻灯片上，但真实程序是三维的。

做法是递归细分：星体稀疏的象限不再细分，密集的区域继续往下分，直到叶子节点。这种结构的便利在于，找附近的东西通常只需往树上走几层，看看兄弟或表亲节点。

因为星体在移动，可能移出原象限，所以每个时间步都要重建这棵树，然后遍历所有节点计算合力，再更新属性。

1.4 本章小结

两个例子给出了并行的两种地形：规则稠密网格与稀疏不规则图。

规则与不规则结构对应两种并行策略

规则稠密结构可以放心按几何形状切分，代价仅为边界通信。不规则结构必须先建立索引结构（树或图），否则切分出去的任务之间负载相差几个数量级。判断策略前先判断数据是规则还是不规则，这一步在分解之前就该完成。

前者容易平均切分，后者需要树结构才能找到有意义的工作划分。后面的方法讨论会交替用这两个例子验证，因为它们暴露的问题并不相同——稠密网格考负载均衡与通信边界，稀疏图考结构重建与不规则负载。

2 并行化的四个步骤

有了例子，讲者给出通用流程。这四个步骤是分析并行程序的基本框架。

2.1 总览

把串行程序变成并行程序，需要依次回答四个问题：哪些工作可以并行、这些工作如何分给各处理器、各处理器如何正确协作、这些逻辑线程最终落到哪些物理单元。每个问题对应一步：分解、分配、编排、映射。

[4]视频画面时间区间：00:08:30–00:08:45。若为裁剪图，时间区间仍对应原视频画面。

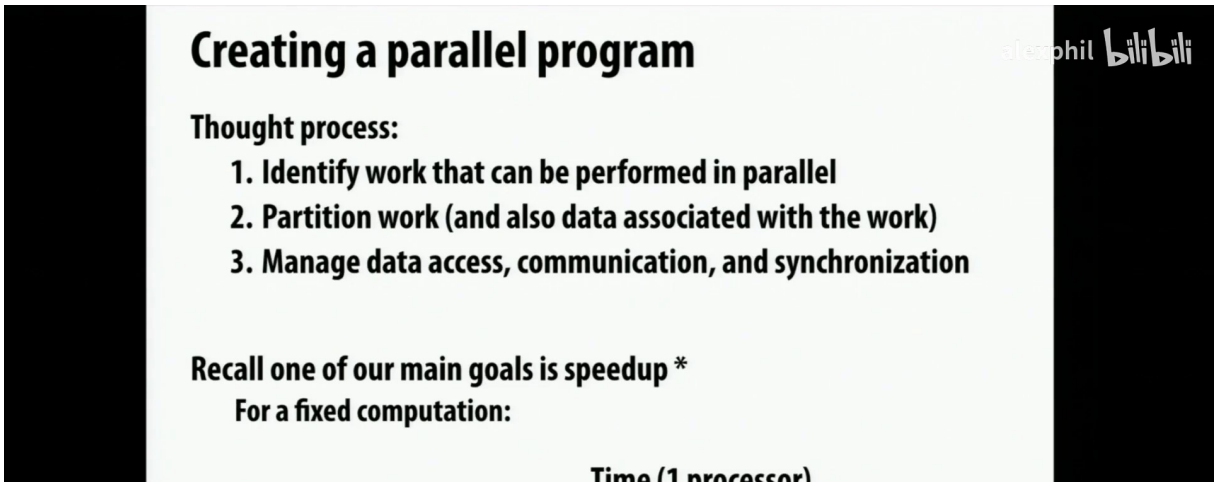


图 5: 把串行程序变成并行程序的思考步骤[5]

讲者用一张图把这层关系画出来：最上面那一整块是需要完成的计算，在单处理器上它是一个整体。并行化就是把它切成小块，而且块数至少要等于处理器数——少于处理器数，必然有处理器无事可做。

任务数与处理器数的关系可以直接写出来：

$$n_{\text{task}} \;\geq\; n_{\text{proc}}, \qquad \text{理想取 } n_{\text{task}} \gg n_{\text{proc}}$$

- n_{task}：分解出的任务（工作块）数量；
- n_{proc}：可用的处理器数量；
- 前者小于后者时，必然有处理器分不到任务而空闲。

这些工作块通常称为任务（task）。第一步就是找出这些自然的工作块。

2.2 分解

分解是纯粹的心智活动：想清楚这项工作有哪些自然的切分方式。

在海洋模拟里，二维数组的每个点都需要计算，因此每个元素是一个潜在任务。任务是独立才构成并行来源——如果一个任务必须等另一个算完才能开始，它们之间不是并行关系，而是串行的。

[5] 视频画面时间区间：00:11:00–00:11:15。若为裁剪图，时间区间仍对应原视频画面。

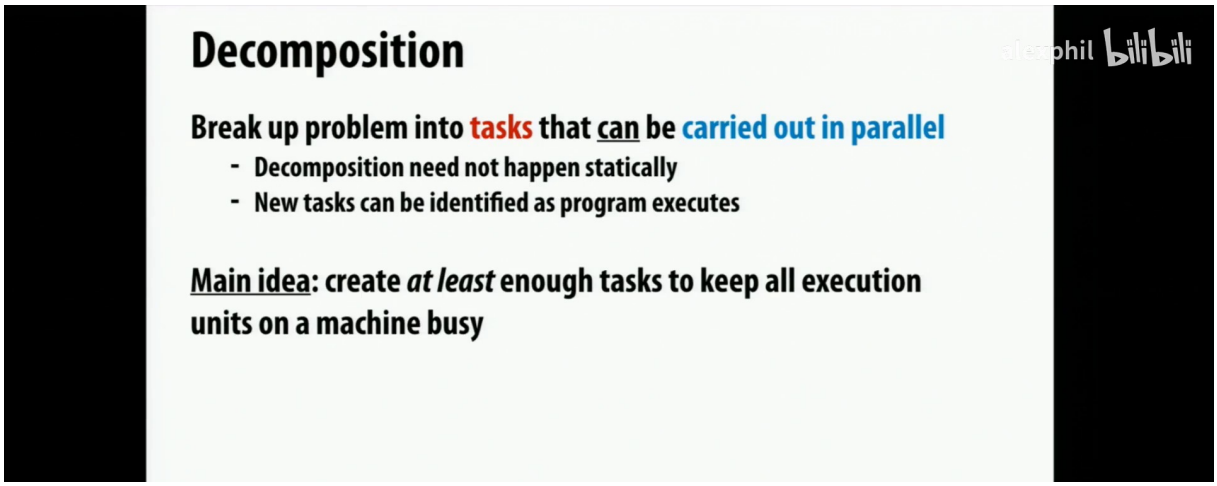


图 6: 分解：把问题拆成可并行执行的任务[6]

这一步几乎总是由程序员完成。理想情况下希望编译器或工具自动推断，但

不要指望编译器替你分解

自动并行化研究已进行多年，成果集中在高度规整的循环嵌套上。一旦出现指针别名、间接寻址或不规则数据结构，编译器无法证明迭代独立，只能放弃。实际项目中分解的合理粒度仍要由人根据算法语义决定。

讲者指出这是很难的问题——他自己也做过这方面的研究，成果有限，因为依赖关系难以整体把握。

2.3 任务粒度的选择

任务数量上的规则很清楚：至少要有线程数那么多，而且通常希望更多。原因是你很难一次就切得恰到好处，任务大小未必相等，多留一些任务才有调配余地。

但也不能走极端。任务切得太小，管理它们本身就会产生开销。因此一个合理的回答是：在星系模拟里，每颗星体是一个任务；在海洋模拟里，可能是单个元素，也可能是整行、整列——讲者说这取决于后续考虑。

粒度选择	优点	代价
任务过少	管理开销低	无法占满处理器，负载不均
任务 = 处理器数	静态即可分配	任何不均都直接变成等待
任务远多于处理器	便于动态均衡	每个任务有调度开销
任务极小	均衡极好	调度开销可能超过计算本身

表 1: 任务粒度与两项成本的关系

[6] 视频画面时间区间：00:16:30–00:16:45。若为裁剪图，时间区间仍对应原视频画面。

2.4 本章小结

分解回答” 有哪些独立工作”，规则结构可按几何切分，不规则结构需先建索引。任务的粒度要满足数量远多于处理器、又不能小到管理开销反客为主。这一步只能由人完成。

分解只确定工作的形状，还没决定谁来做。把任务实际落到处理器上，是下一步的工作，也是本课第一讲演示中负载不均教训的直接延续。

2.5 分配的目标

分配的目标是让每个处理器拿到大致相同的工作量。讲者回到第一堂课的演示：四个志愿者加十六个数，他故意让一个人分到八个数字，结果另外三个人早早完成，全体等那最后一个人。

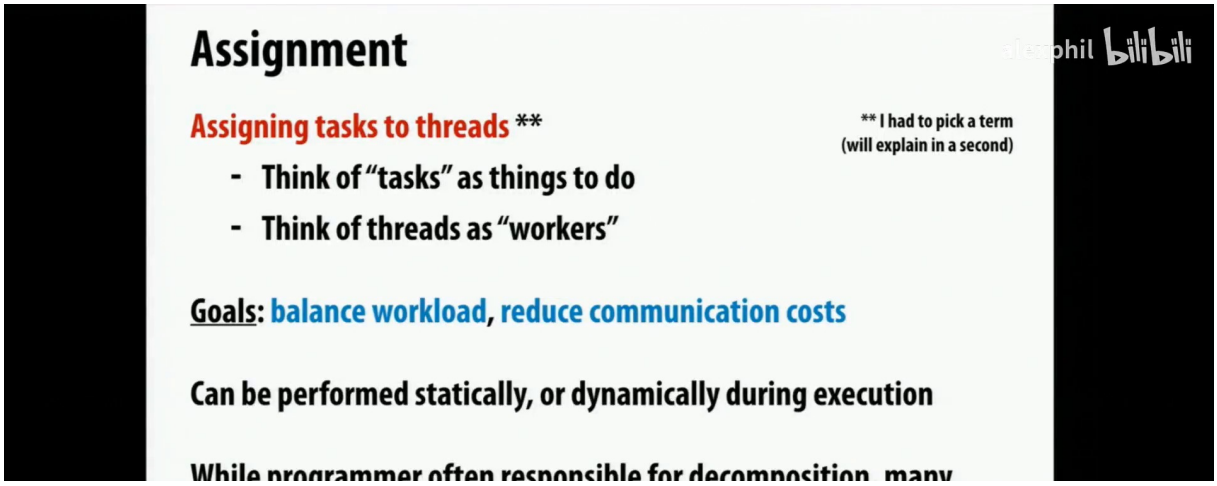


图 7: 分配：把任务分给线程[7]

第二次他们重新分配，每人四个，情况好得多。这个对比就是本章第 4 节要展开的静态与动态两种分配方式。

2.6 编排

分解与分配只是在切分工作量。真正让线程协作起来的那一步叫编排，它的细节高度依赖编程抽象。

[7]视频画面时间区间：00:26:00–00:26:15。若为裁剪图，时间区间仍对应原视频画面。

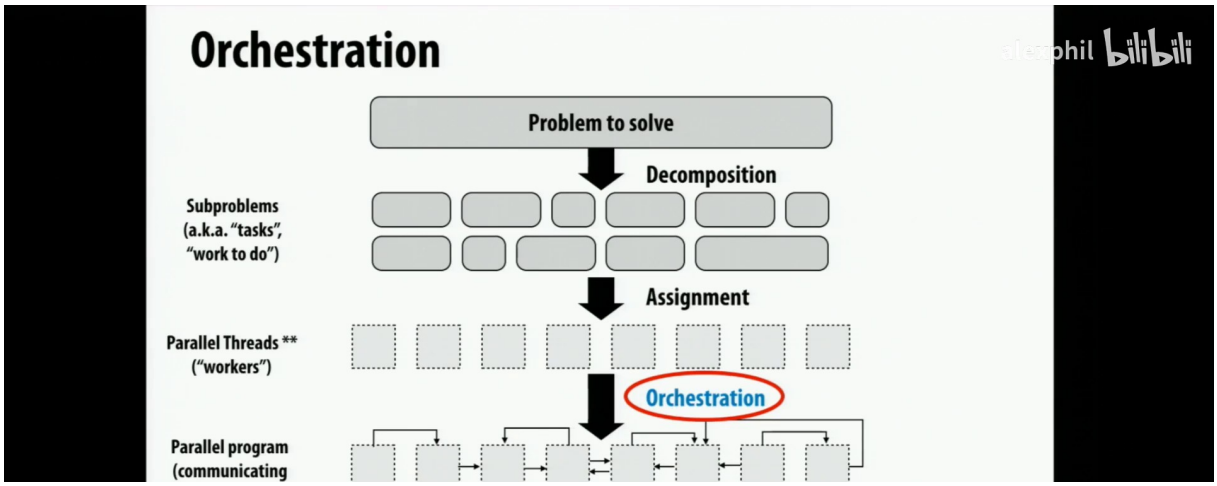


图 8: 编排：在分解出的子问题之间组织协作[8]

这是四个步骤里最有分化的一步：共享地址空间、消息传递、数据并行三种抽象下的编排代码看起来完全不同。因此本讲后面会用同一个网格求解器在两种模型下各写一遍，直接对比。

2.7 映射

最后一步是把逻辑线程映射到物理硬件单元。多数情况下程序员让系统自行决定，但有些场景需要更小心。

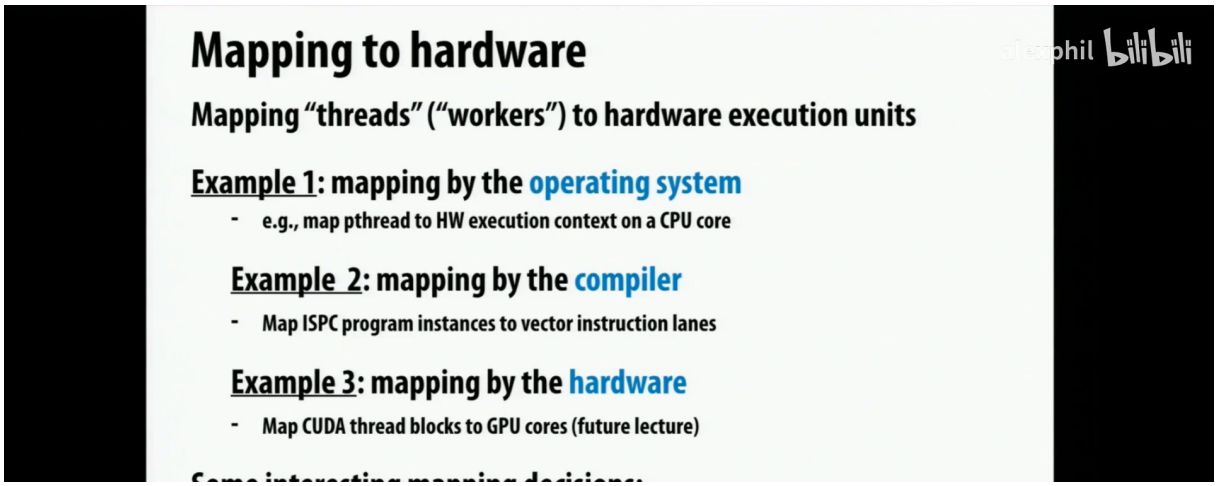


图 9: 映射：把线程（工作单元）对应到硬件的执行单元[9]

讲者给出的例子是：如果互连只允许与近邻通信，那就需要确保相邻的逻辑线程被映射到物理上相邻的单元，否则每次通信都要绕远。

2.8 本章小结

分配把任务落到处理器上，目标是让每个处理器拿到相近的工作量。四步之中，分解与分配决定理论上限，编排与映射决定实际能拿到多少。

[8] 视频画面时间区间：00:35:30–00:35:45。若为裁剪图，时间区间仍对应原视频画面。
[9] 视频画面时间区间：00:38:30–00:38:45。若为裁剪图，时间区间仍对应原视频画面。

四步中最容易出错也最容易被低估的是分配，因为它直接对应第一课演示里那个负载不均的教训。下一章先量化一件事：即使分配完美，还有另一个因素给加速设上限。

3 Amdahl 定律：串行部分设下的天花板

上一讲已经用演示说明加速会打折。这一节给出定量的上界，并用一个具体例子算清楚。

3.1 一个常见错误

讲者先指出一个在研究并行时常见的失误：看到某段代码容易并行，就格外兴奋地去优化它，而忽略那些难以处理的段落，还以为这是明智之举。

为了说明这并不是明智之举，他请学生回顾 Amdahl 定律。

3.2 图像处理的例子

设想一个图像处理算法分两步：把每个像素的亮度加倍，然后计算处理后所有像素的平均值。

第一步没有依赖，每个像素独立处理，容易并行。第二步是归约，有依赖关系，性质类似第一课全场加总课程数的演示。

顺序执行时，两步各要访问 N^2 个元素，因此总耗时正比于 $2N^2$。

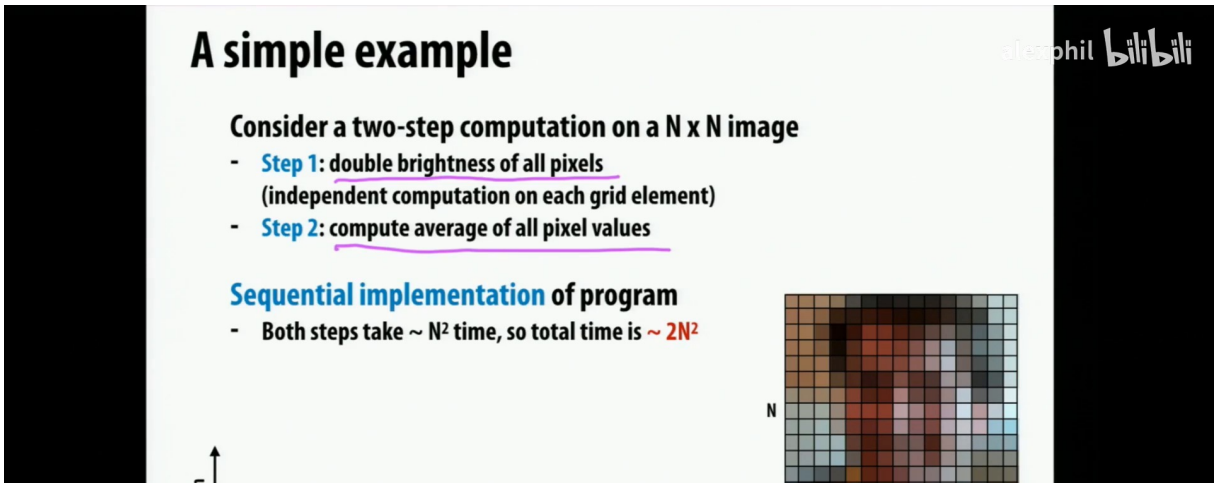


图 10: 两步计算的简单例子：先加倍亮度，再求平均[10]

3.3 只并行第一步的后果

在 P 个处理器上，第一步可以并行达到 P 倍加速：

$$T_P = \frac{N^2}{P} + N^2, \qquad T_1 = 2N^2$$

- T_1：单处理器总耗时，正比于 $2N^2$；
- T_P：P 个处理器上的总耗时；
- N^2/P：第一步在 P 个处理器上的耗时；

[10] 视频画面时间区间：00:19:30–00:19:45。若为裁剪图，时间区间仍对应原视频画面。

- N^2：第二步未并行，耗时不变。

讲者用一张时间轴图说明这个结果：横轴是时间，纵轴是正在被利用的并行度。顺序版本全程是 1；并行版本里紫色那一大段抬高到 P，但绿色那一段仍然停在 1。

于是当 P 趋于无穷时：

$$S_\infty = \lim_{P\to\infty} \frac{2N^2}{N^2/P + N^2} = 2$$

- S_∞：处理器数量趋于无穷时的加速比；
- 分子 $2N^2$：顺序基准；
- 分母中仅剩的 N^2：未被加速的第二步，它成了整个加速比的瓶颈。

投入无限多处理器，最多只能快两倍。这个数字很有说服力：一半的计算不可并行，就吞掉了一半的潜在收益。

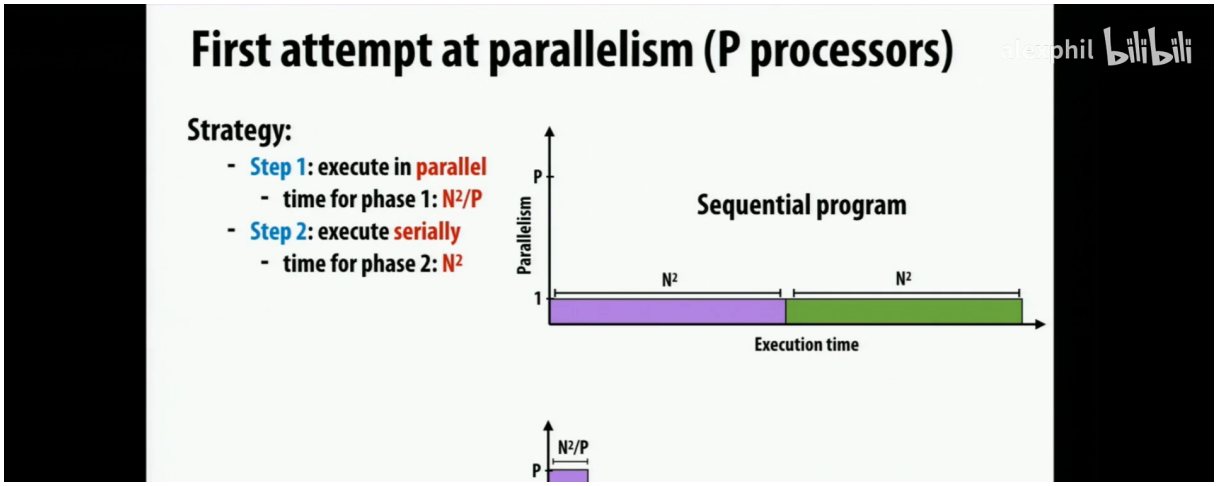


图 11：只并行第一步的第一次尝试与其时间轴[11]

3.4 把第二步也并行：部分和

第二步虽然不是天然独立，但有一个通用技巧：让每个处理器先算出自己那部分数据的部分和。这一步彼此独立，可以并行。

然后才把 P 个部分和合并成一个总和。合并只能串行进行。

[11] 视频画面时间区间：00:20:30–00:20:45。若为裁剪图，时间区间仍对应原视频画面。

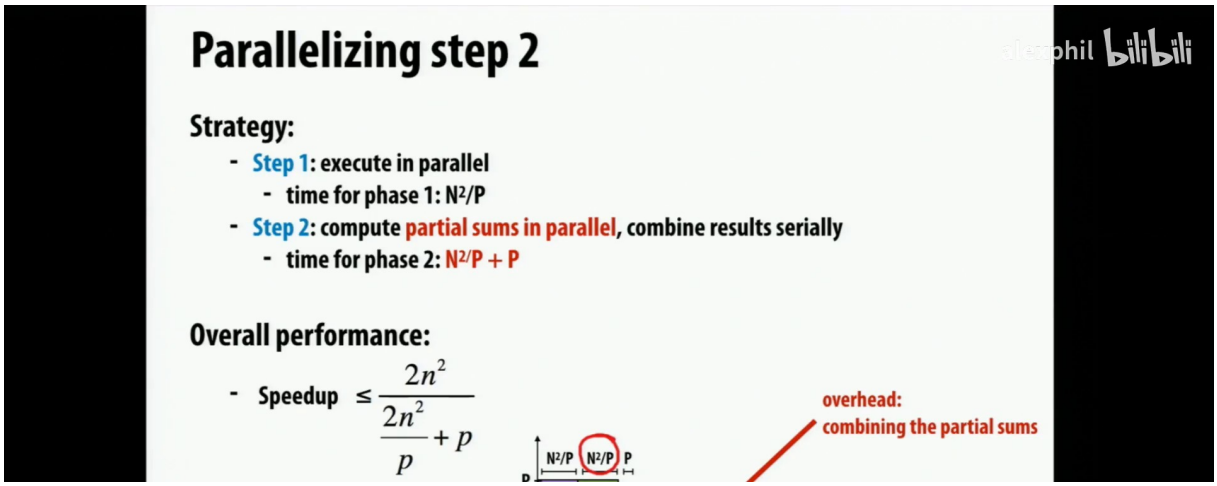


图 12: 并行化第二步：各处理器算部分和，再串行合并[12]

关键观察是新的串行部分宽度只有 P 个部分和，而不是 N^2 个元素。合并这 P 个部分和只需逐个相加，串行代价从 N 的平方规模降到 2 个量级的跨度。因为 N 通常远大于 P，这段串行开销可以被摊薄到几乎可以忽略。讲者的说法是会得到接近完美的加速。

3.5 定律的一般形式

讲者随后把结论推广。设 S 是总工作量中本质上串行的比例，则最大加速比为：

$$S_{\max} = \frac{1}{S + \dfrac{1 - S}{P}}$$

- S：总工作中无法并行的比例，$0 \le S \le 1$；
- P：处理器数量；
- 当 $P \to \infty$ 时，上界趋近 $1/S$。

代入具体数字就明白它有多严厉。若只有九成的代码可以并行，也就是串行占一成，那么无论投入多少处理器，加速都超不过 10 倍。即使可并行的部分提高到百分之九十九，上限也只有 100 倍。这两个数字来自把比例直接代入上界公式：串行比例 0.10 时上界为 10，串行比例 0.01 时上界为 100。投入再多处理器也突破不了这两个上限。

可并行比例	串行比例 S	加速上限 $1/S$
90%	0.10	10 倍
99%	0.01	100 倍
99.9%	0.001	1000 倍

表 2: Amdahl 定律的量化含义：串行比例决定加速天花板

[12]视频画面时间区间：00:23:00–00:23:15。若为裁剪图，时间区间仍对应原视频画面。

先算上界，再决定投入

Amdahl 定律给出的是”投入更多处理器也拿不到”的严格上界。在规划并行化时先用它估算可达到的上限，比先写完代码再发现收益封顶更省力。若上界远低于投核计划，应该先减少串行部分（如把归约改成部分和），而不是继续增加处理器。

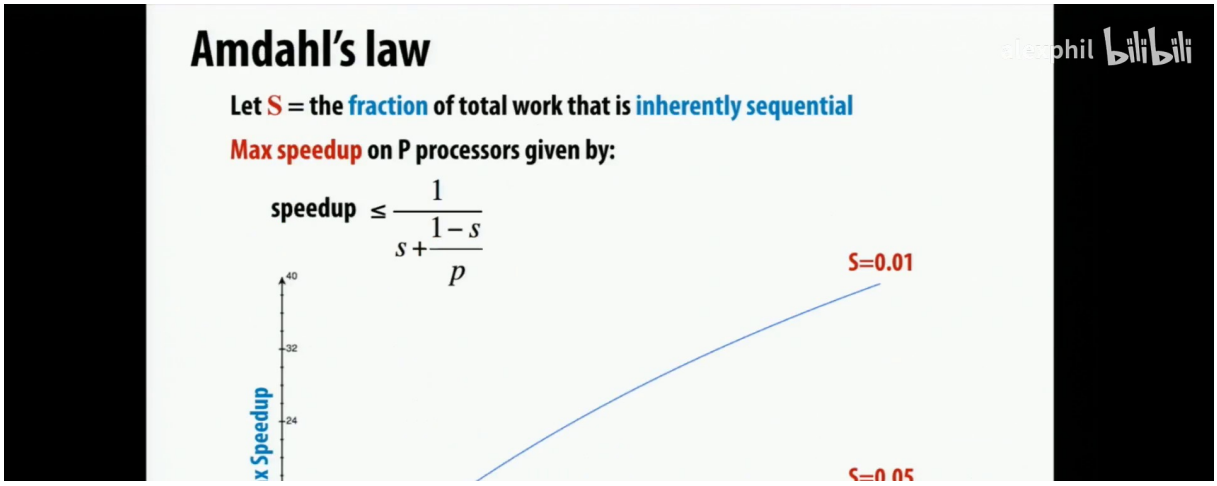


图 13: Amdahl 定律：设 S 为固有串行比例，最大加速为 $1/(S+(1-S)/P)$[13]

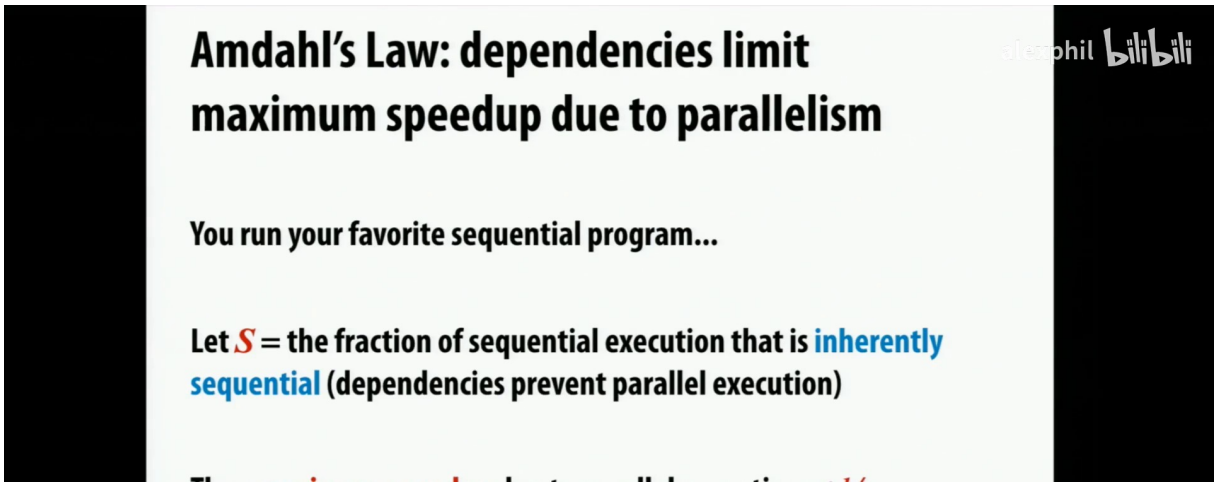


图 14: 依赖限制最大加速：Amdahl 定律的直观表述[14]

3.6 本章小结

Amdahl 定律把”加速会打折”量化为一个硬上界：串行比例 S 决定了加速最多为 $1/S$，与处理器数量无关。图像处理的例子给出两个具体数字——只并行第一步，极限是 2 倍；改成分部分和加串行合并后，串行宽度从 N^2 降到 P，几乎可以忽略。

这条定律回答的是”值不值得并行某一部分”。而”实际能拿到多少”还取决于分配与编排，那才是下一章的内容。

[13] 视频画面时间区间：00:23:30–00:23:45。若为裁剪图，时间区间仍对应原视频画面。

[14] 视频画面时间区间：00:18:30–00:18:45。若为裁剪图，时间区间仍对应原视频画面。

4　分配的两种策略：静态与动态

Amdahl 定律假定分配是完美的。现实中分配方式本身就会决定性能，而分配策略可以分为静态与动态两大类，二者的开销来源恰好相反。

4.1　静态分配与它的问题

静态分配在编译期或启动时就把任务固定交给各线程，运行期不再调整。它简单、无调度开销，但对工作量分布的估计有很强依赖。

静态分配在编译期或启动时就把任务固定交给各线程，运行期不再调整。它简单、无调度开销，但对工作量分布的估计有很强依赖。

4.2　一个具体写法与它暴露的两个问题

讲者展示了一段用 pthreads 做静态分配的代码：把输入数组对半分，创建一个子线程处理一半，父线程处理另一半。

这里有两个问题。

把线程数写死等于放弃可移植性

按数组对半分并创建固定数量的线程，在核心数变化时要么浪费硬件，要么超出可用核数。可移植的写法应查询运行时的可用线程数，或把工作切成远多于线程数的任务再交给调度器分配。

第一，代码硬编码只创建两个线程；如果机器有四核，就有一半硬件闲置。第二，即使只有两核，这种对半分也不一定均等：如果某半边的计算量明显更大，先完成的那一半就要空等。

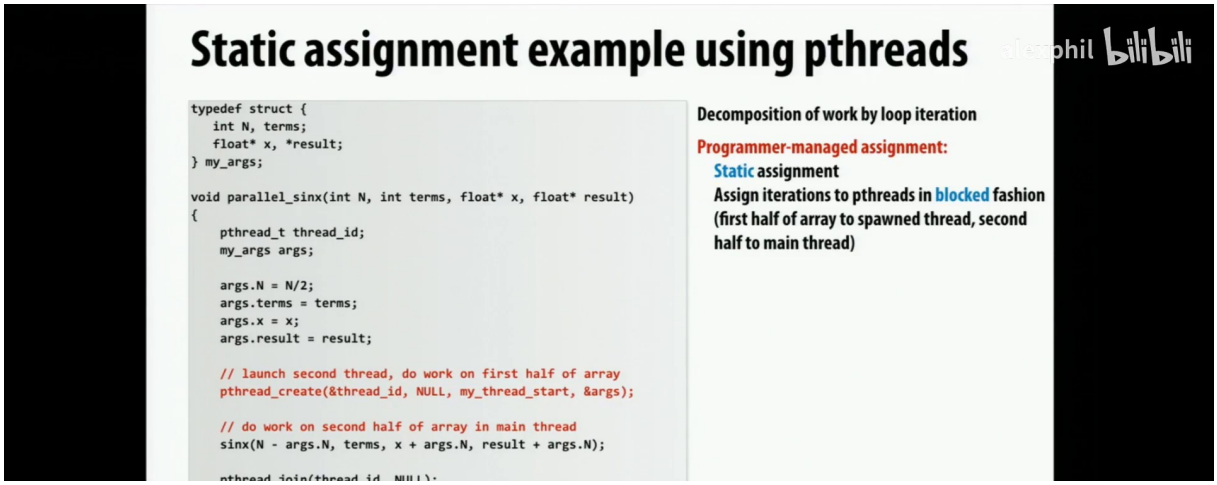


图 15：用 pthreads 做静态分配：手工按循环迭代切分[15]

ISPC 中的静态分配写法更简洁，由 forEach 把迭代分给实例。但负载均衡的问题依然存在——只是被语言包装起来，并没有消失。

[15] 视频画面时间区间：00:30:00–00:30:15。若为裁剪图，时间区间仍对应原视频画面。

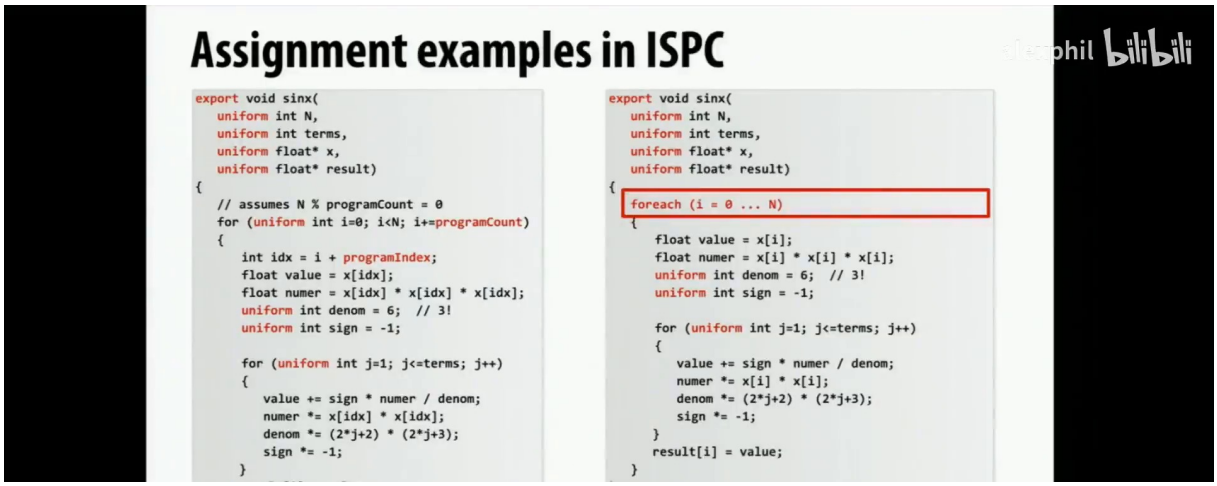


图 16: ISPC 中的分配示例：由 forEach 交出迭代分配[16]

4.3 本章小结

静态分配没有运行期开销，代价是要求事先估准工作量，且线程数写死时无法适应机器核心数。对均匀且稳定的工作，它是最省的选择；一旦工作量分布不可预测，这份节省会以空等的方式还回去。

要让程序自动适应变化的负载，就需要把分配推迟到运行期，这正是下一章的做法。

5 动态分配：用队列换均衡

5.1 工作队列的做法

解决思路是把工作放进一个队列，谁空了谁去取下一件。

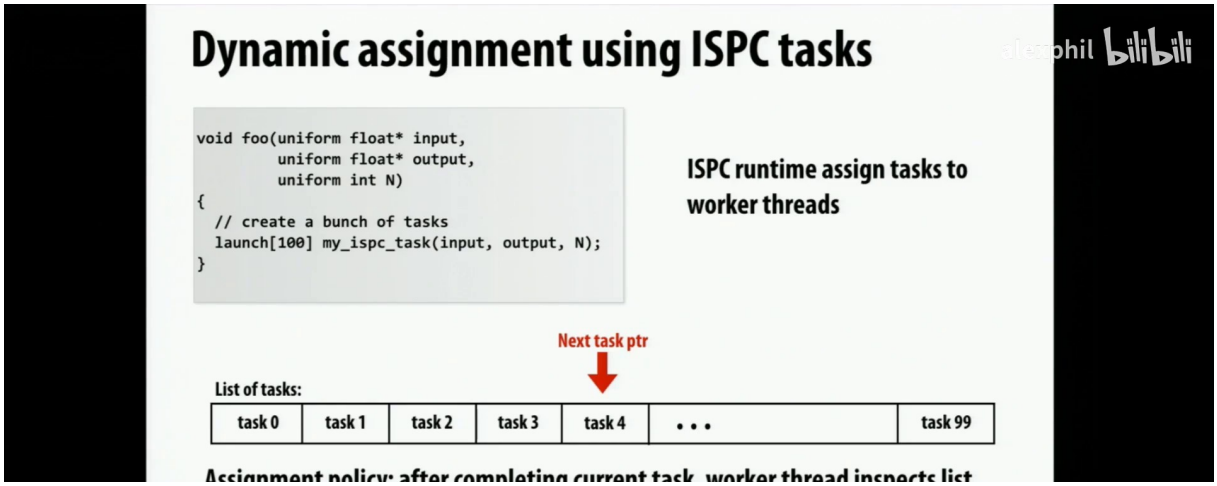


图 17: 用 ISPC task 做动态分配：运行时把任务派给空闲的执行体[17]

[16] 视频画面时间区间：00:28:30–00:28:45。若为裁剪图，时间区间仍对应原视频画面。

[17] 视频画面时间区间：00:33:30–00:33:45。若为裁剪图，时间区间仍对应原视频画面。

ISPC 的 task 就是这样一种机制：把一堆工作块交出去，系统内部维护一个指向下一个待派任务的指针。开始时每个核心拿一个，谁先完成谁回去取下一个。

讲者请学生评价这种方法的优劣，得到的正是它的本质权衡。

静态与动态两种分配的运行期总开销可以用同一形式比较：

$$T_{\text{static}} = \frac{W}{P} + \delta, \qquad T_{\text{dynamic}} = \frac{W}{P} + \frac{W}{s} \cdot c_{\text{queue}}$$

- W：总工作量；
- P：处理器（线程）数量；
- δ：静态分配的负载不均损失；
- s：单个任务的规模，W/s 即任务个数；
- c_{queue}：一次取任务（含加锁）的开销。

静态分配把成本压在 δ 上，动态分配把它转移到任务数乘以每次取任务的代价。这也是任务不能切得太细的原因：s 越小，第二项越大。

动态分配的收益与代价同源

收益：负载自动均衡，某个任务耗时很长也不影响整体，因为其余处理器会继续取新任务。代价：取任务本身要访问共享队列，通常需要加锁保护，因此多个处理器同时取任务时必须串行化。任务切得越细，这份开销占比越高。静态分配没有这项开销，但要求事先估准工作量。

5.2 本章小结

动态分配用共享队列换取自动均衡：某件任务耗时很长也不影响整体，因为其余处理器会继续取新任务。代价是每次取任务都要访问共享队列，通常还需加锁，多处理器同时取任务时必须串行化。任务切得越细，这份开销占比越高。选择依据是工作量的可预测性与任务粒度。

6 编排：用红黑格序消除数据竞争

前面三步处理的是” 分多少、分给谁”。编排处理的是” 怎么配合”，而这个问题的难度在这门课里最容易被低估。

6.1 网格求解器的串行版本

讲者引入一个具体的求解器：在 $(N+2) \times (N+2)$ 的网格上求解偏微分方程。外层是收敛循环，每次比较新旧值，把全部元素差值的绝对值求和，低于阈值就停止。

内层是对二维数组的遍历，每个元素取自身与上下左右四个邻居的平均，写回原位置。因为要计算新旧之差，必须先记住旧值。

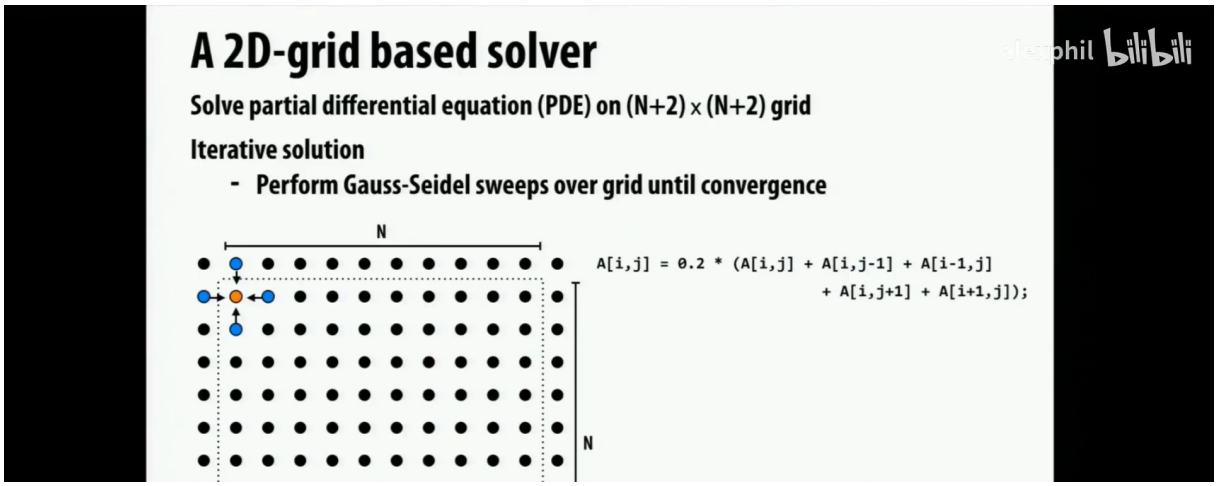


图 18: 二维网格求解器：顺序执行的伪代码[18]

6.2 依赖在哪里

如果直接把这个双重循环并行化，会发生什么？讲者让课堂思考依赖关系，答案是数据竞争：每次运行可能得到不同结果。

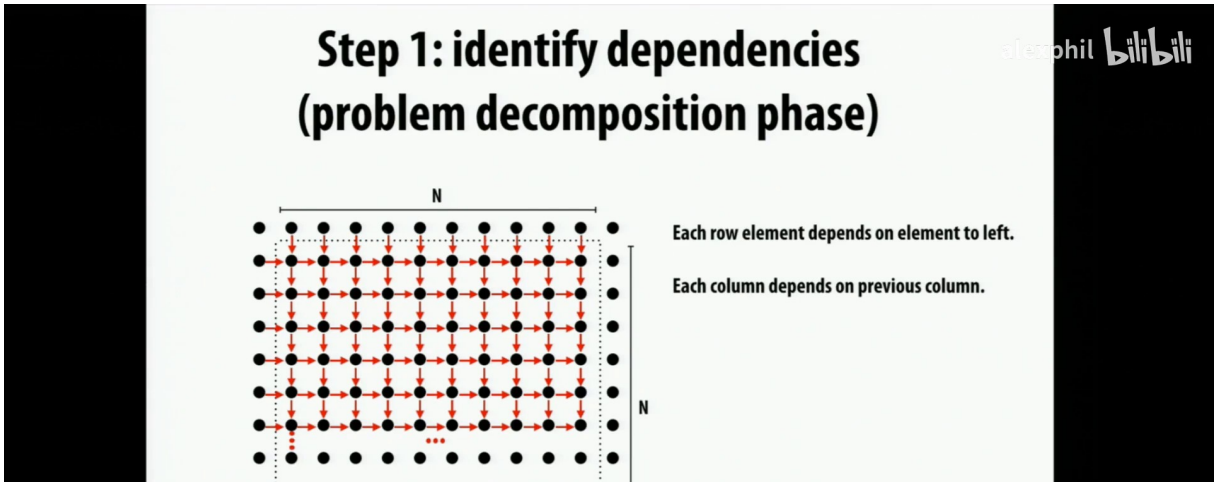


图 19: 第一步：识别依赖关系——每个元素依赖它的邻居[19]

原因很直接：写第 j 个元素时读的是它左右的邻居，而邻居可能正被另一个线程更新。于是读到的值取决于执行时序。讲者用”红色箭头”表示这些跨迭代的依赖，图上密密麻麻。

就地更新加并行等于非确定性

当写操作的目标与读操作的来源在同一数组内，并行执行的结果就取决于线程交错顺序。这类错误的危险之处在于它不总是报错——程序可能多数时候算对，偶尔给出略微不同的数值，极难复现。

[18] 视频画面时间区间：00:43:30–00:43:45。若为裁剪图，时间区间仍对应原视频画面。
[19] 视频画面时间区间：00:46:30–00:46:45。若为裁剪图，时间区间仍对应原视频画面。

6.3 双缓冲与它为什么被拒绝

最直接的修法是准备两份矩阵，一份读一份写，下一轮交换角色。这样读的永远是不变的副本，竞争消失。

但讲者说科学计算者拒绝这么做，理由是把内存填满数据是他们的重要目标，甚至不愿意砍掉一半。他的评语是可以争论这合不合理，但这就是他们的想法。

双缓冲为何在科学计算中不流行

双缓冲需要的额外内存正比于整个数据体。对受内存容量限制的大规模模拟，这直接减半可求解的问题规模。因此这个做法在图形或游戏里很常见，在科学计算里常被视为不可接受。替代方案必须在不复制全部数据的前提下工作。

6.4 红黑格序

替代方案借用了国际象棋棋盘的想法。

把网格元素按棋盘模式着色：相邻元素颜色不同。然后分两阶段更新：先只更新所有红格，再只更新所有黑格。

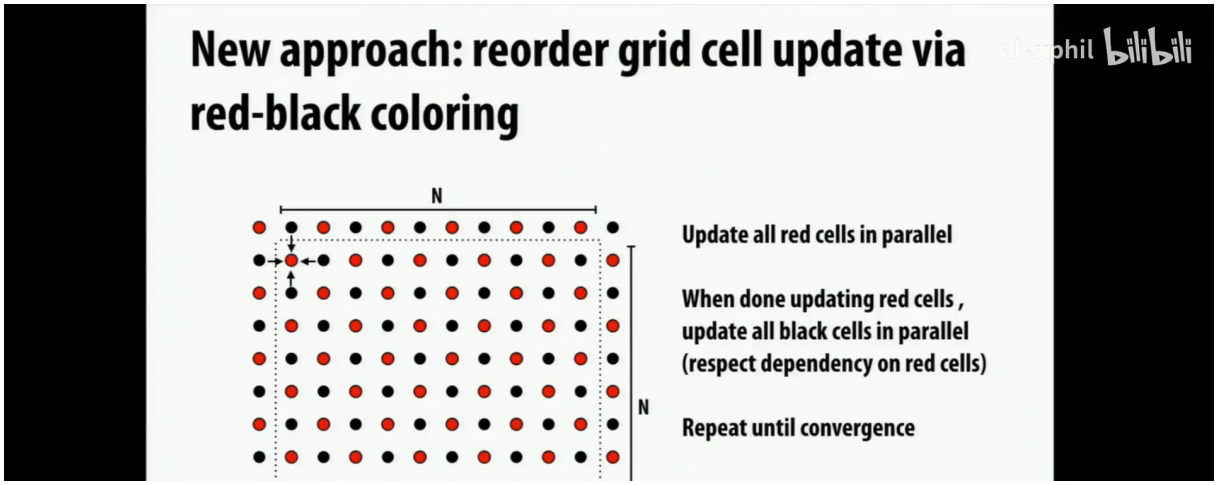


图 20: 红黑格序：分两阶段交替更新红格与黑格[20]

这个安排的效果等同于双缓冲：更新红格时，它读的是自身与黑格邻居，而黑格此刻不被修改，所以读到的值稳定。然后交换角色。

[20] 视频画面时间区间：00:51:30–00:51:45。若为裁剪图，时间区间仍对应原视频画面。

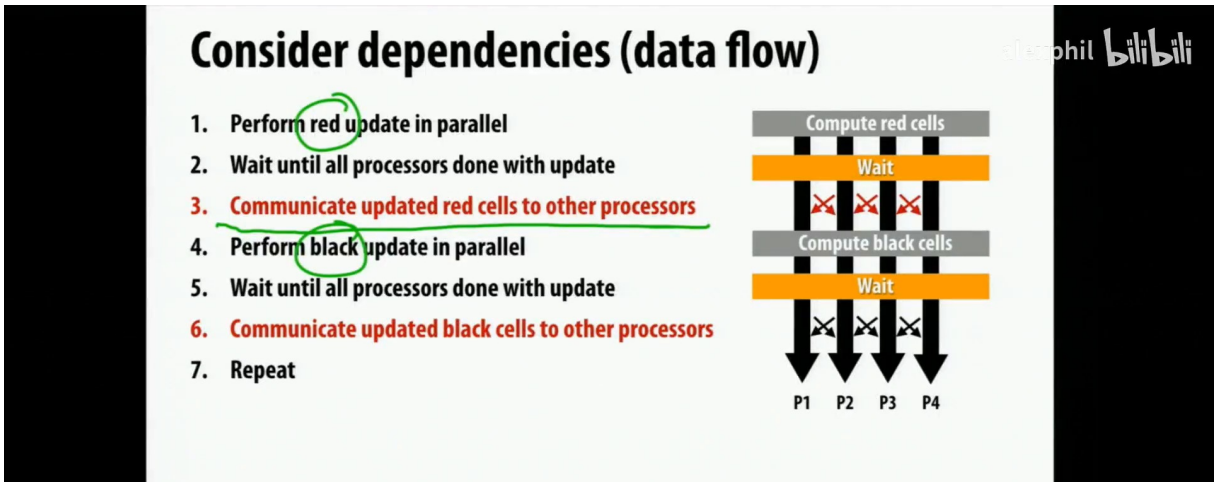


图 21：按红黑格序重排后的数据流：两阶段依赖清晰[21]

讲者坦承一个代价：结果与顺序版本并不完全相同。

确定性不等于与顺序版本等价

红黑格序每次运行结果一致，这是确定性；但这个结果与逐元素就地更新的顺序版本不同。对迭代收敛的物理模拟，这点差异通常可接受；对要求逐位复现某个参考实现的场景，则不能直接替换，必须先验证误差在容许范围内。

但对这类模拟通常足够接近，而且它是确定的——数据竞争消失，每次运行结果一致，也不需要复制全部元素。

6.5 阶段之间需要同步

学生提出一个关键追问：更新黑格时，红格已经改过了，黑格的邻居确实是被更新过的新值。

讲者的回答是：那正是我们想要的，但前提是必须确保所有对红格的更新都已正确写出并对所有人可见。这需要两阶段之间的一次同步，也就是屏障。

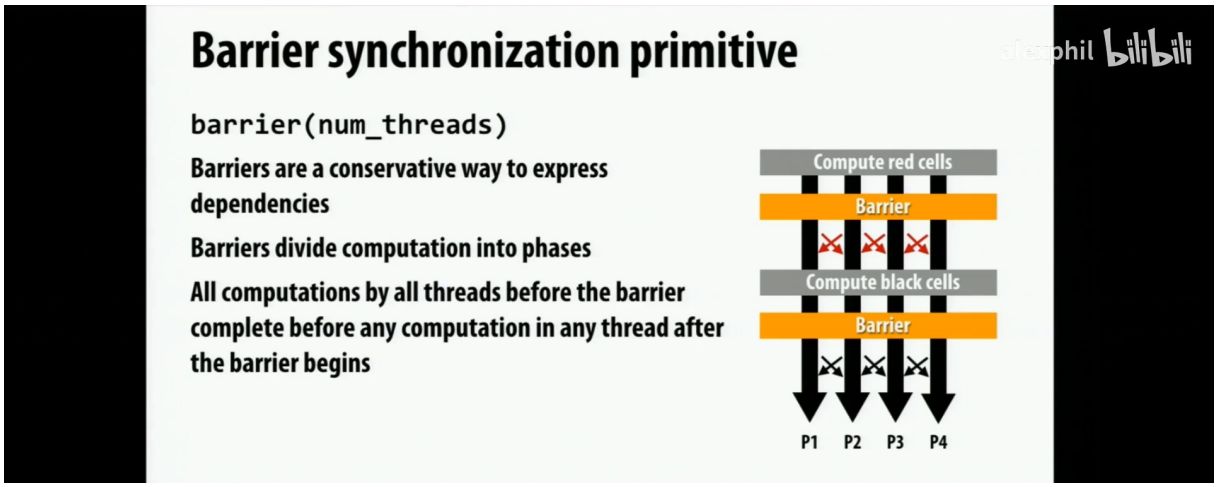


图 22：屏障同步原语：所有线程在此汇合[22]

[21] 视频画面时间区间：00:56:00–00:56:15。若为裁剪图，时间区间仍对应原视频画面。

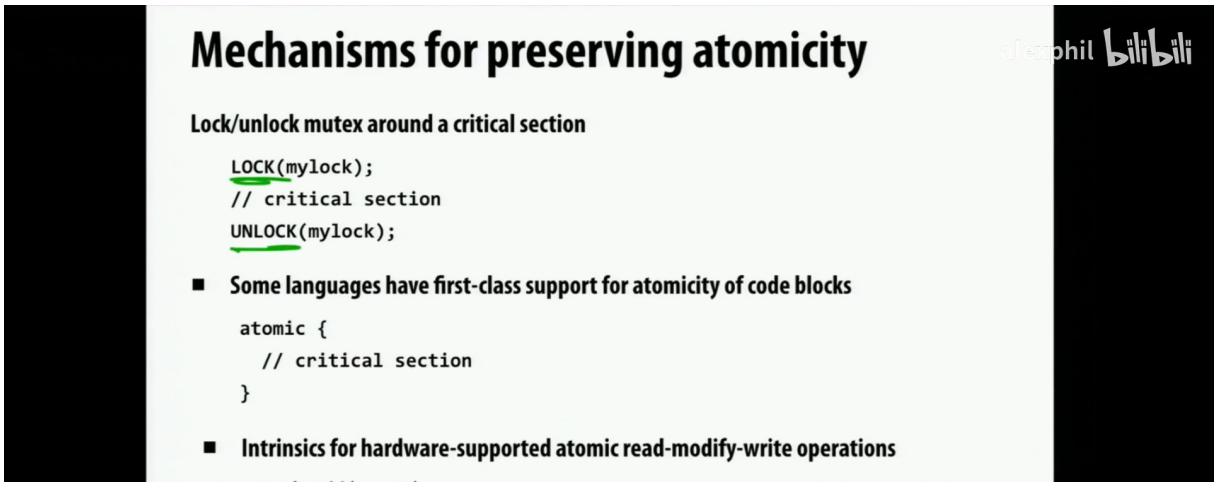


图 23: 保证原子性的机制：临界区加锁与解锁[23]

6.6 屏障的代价与更精确的依赖

讲者指出屏障是个重量级的工具：它会强迫全部线程彼此追上进度。

屏障表达的是最粗的依赖

一个屏障表示”所有线程都必须等我”，它保守地覆盖了全部依赖。当实际依赖只涉及局部数据时，用屏障会让本可继续工作的线程被迫停下。屏障越频繁，其开销占比越高；这也是后续要考察减少同步次数的主要原因。

用一¨刀切的¨方式解决依赖固然简单，但代价是失去了细粒度调度的可能。
如果知道只需等某个特定位置更新完，就可以更精确地等待，而不必让所有线程同步。

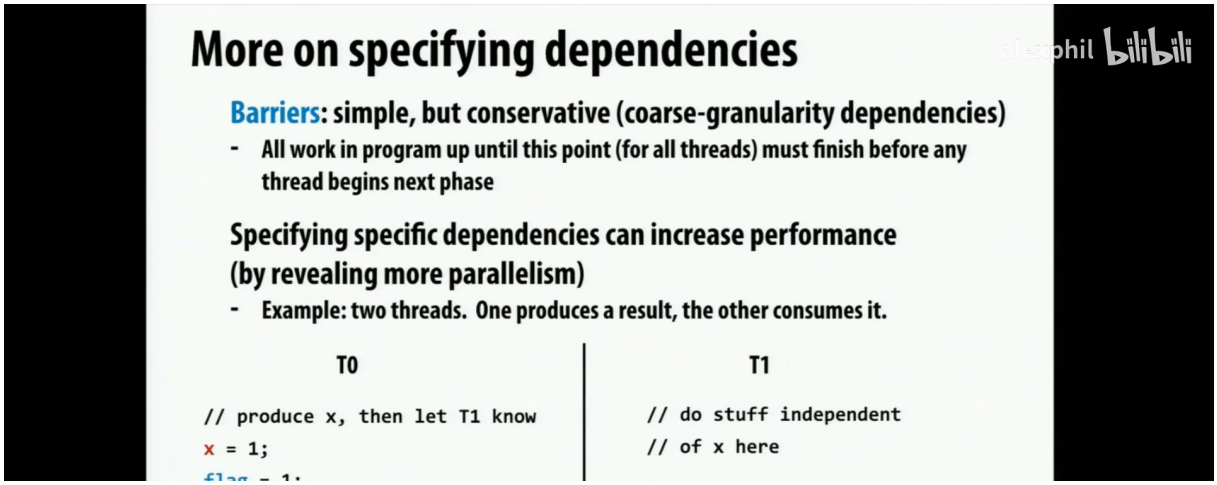


图 24: 更精确地表达依赖：屏障简单但保守[24]

[22] 视频画面时间区间：01:07:30–01:07:45。若为裁剪图，时间区间仍对应原视频画面。
[23] 视频画面时间区间：01:06:00–01:06:15。若为裁剪图，时间区间仍对应原视频画面。
[24] 视频画面时间区间：01:11:00–01:11:15。若为裁剪图，时间区间仍对应原视频画面。

不要用普通内存位置做同步

讲者在展示这种细粒度同步时明确警告：按那种写法不能直接用，中间必须有一个栅栏（fence），否则完全不能工作。用普通内存读写充当同步信号，在编译器与硬件重排面前是不可靠的。这个概念会在后面讲内存一致性模型的课上专门展开。

6.7 用多份副本减少屏障

讲者还展示了一个实用技巧。如果某个标量（例如差值累加量 diff）在多个迭代间被复用，可能出现还没用完就被下一轮覆盖的情况。

解法是准备三份副本，按下标取模交替使用。因为 diff 只是标量，三份副本几乎不占空间。这样每个线程始终指向正确的那份，于是本阶段只需要一个屏障。

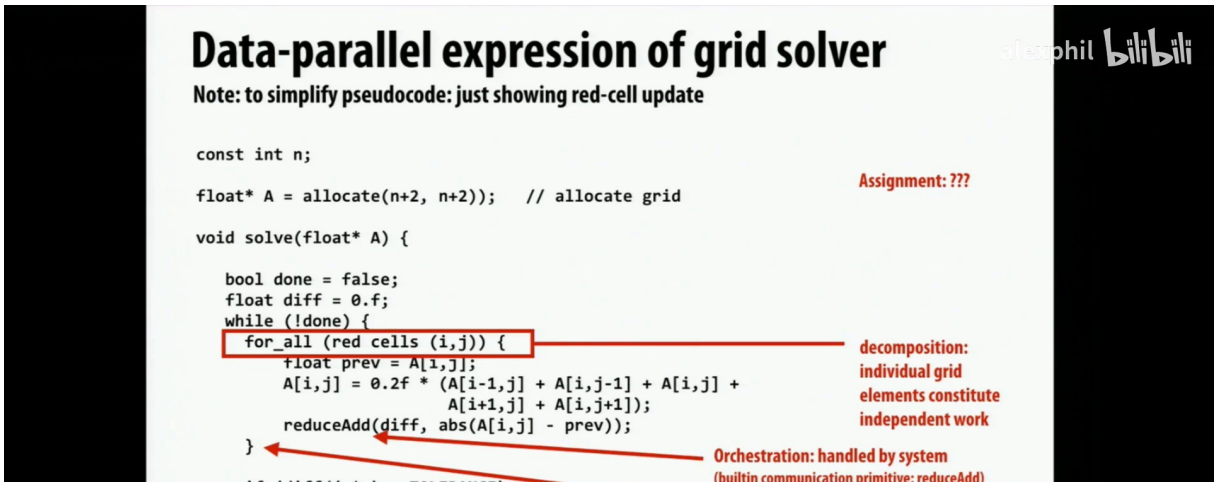


图 25: 网格求解器的数据并行表达[25]

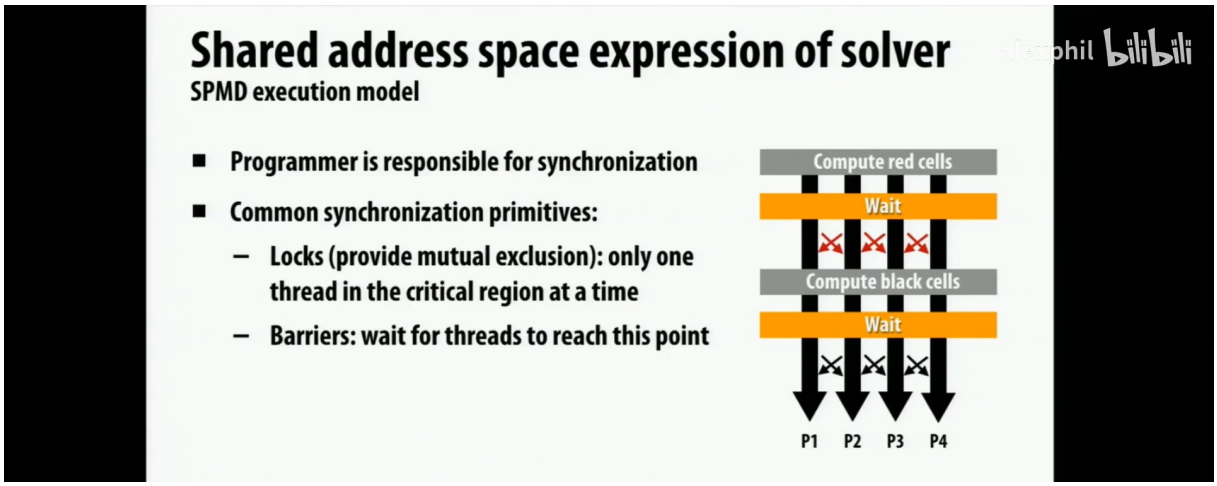


图 26: 网格求解器的共享地址空间表达：SPMD 执行模型[26]

6.8 本章小结

就地更新加并行会产生数据竞争，双缓冲能消除它但在科学计算中不可接受。红黑格序以改变更新顺序为代价，在不复制数据的前提下得到确定性，代价是两阶段之间需要屏障。

这条路径展示了编排的实质：正确性来自显式表达依赖，而不是依赖时序巧合。屏障是最简单的表达方式，也是最保守的，更精确的依赖表达需要配套的内存序保证。

到此为止的编排都建立在共享变量上。换一种抽象，同一份求解器的协作方式会完全不同，这正是下一章要对比的。

7 消息传递：同一个求解器的另一种编排

讲者最后用同一个网格求解器再写一遍，这次用消息传递，让读者直接看到编排的差异。

[25] 视频画面时间区间：00:58:30–00:58:45。若为裁剪图，时间区间仍对应原视频画面。

[26] 视频画面时间区间：01:01:30–01:01:45。若为裁剪图，时间区间仍对应原视频画面。

7.1 通信只能靠消息

在消息传递模型里，通信只通过给其他线程发消息、由对方接收消息来完成。没有共享变量可用。

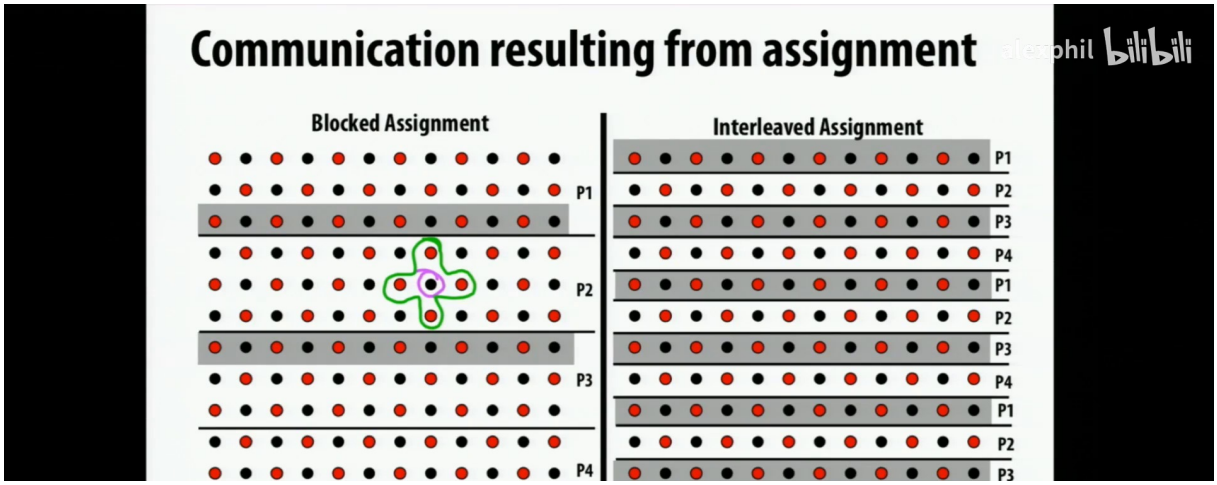


图 27: 不同分配方案产生的通信需求[27]

这意味着共享地址空间里”读一下邻居”这种自然的操作，在这里必须变成显式的发送与接收。分配方式也因此变得重要：阻塞式分配下，每个线程只需要与相邻线程交换边界数据；交叉式分配则会让一次更新需要到处取数。

7.2 死锁与它的解法

讲者展示了交换边界数据时的经典陷阱：如果所有线程都先发后收，就可能互相等待对方腾出缓冲，形成死锁。

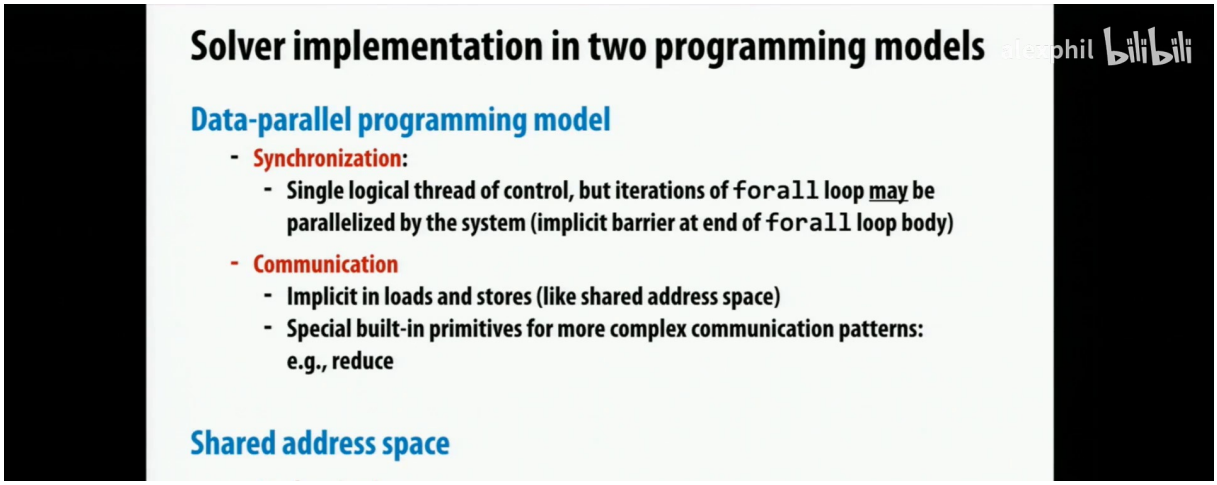


图 28: 同一求解器在两种编程模型下的实现对照[28]

两种常规解法。第一种是按编号奇偶交替：偶数号先发后收，奇数号先收后发，这样总有一半线程在收，不会全部阻塞。第二种是使用非阻塞的发送与接收。

[27] 视频画面时间区间：00:57:00–00:57:15。若为裁剪图，时间区间仍对应原视频画面。

[28] 视频画面时间区间：01:12:30–01:12:45。若为裁剪图，时间区间仍对应原视频画面。

死锁是消息传递的入门门槛

共享地址空间里没有对应问题，因为读写不需要双方同时就位。消息传递里传与收是成对的阻塞操作，顺序不当就会互相等待。写消息传递代码时，必须先确认每个通信对的先后顺序，或者直接使用非阻塞原语。

7.3 本章小结

同一个红黑格序求解器，在共享地址空间里靠屏障表达依赖，在消息传递里靠交换边界数据加奇偶交替（或非阻塞收发）完成。编排的正确性标准变高了：不仅要算对依赖，还要保证通信不成环等待。

这也回到讲者开头提出的对照——编排是四步中唯一随抽象剧烈变化的一步，而它恰好是决定实际性能的关键。

8 总结与延伸

这一讲把”怎么写并行程序”拆成了可检查的步骤，并给出了每一步的失败模式。

讲者的收尾很简洁：我们讨论了编写并行代码的各个步骤——分解、分配，并看了不同的编程模型。接下来两讲会深入这些问题的更多细节。

把全讲串起来，逻辑是这样闭合的。分解决定有多少独立工作，任务数必须不少于处理器数、通常还要更多，但切得太细会带来管理开销。分配决定这些工作如何均衡落下，静态方式没有运行时开销却要求事先估准，动态队列能自动均衡却要付锁竞争。Amdahl 定律在此之上给出一个与处理器数量无关的上界：串行比例 S 把加速封在 $1/S$，图像处理的 $2N^2$ 例子说明只并行第一步最多快两倍。编排决定线程能否正确协作，网格求解器展示了就地更新导致的竞争，以及红黑格序如何在拒绝双缓冲的前提下恢复确定性。消息传递则把同一问题的编排换成显式通信，并引入死锁这一新风险。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 四步中存在”先量化再动手”的顺序。Amdahl 定律应在分解阶段就用来判断某一部分值不值得并行，而不是等写完代码再测。
- 分配与编排的代价相互牵扯。动态分配为了均衡而引入的锁，本身就是一个需要编排的共享数据结构；任务切细会同时提高均衡性和调度开销。
- 红黑格序是被内存容量逼出来的解法。它的存在前提是不许复制数据，这个前提在内存宽裕的场景下不成立，双缓冲反而更简单。
- 编排的正确性分两层。第一层是消除竞争（红黑格序），第二层是保证可见性与顺序（屏障、栅栏）。只做第一层而不做第二层，代码会时对时错。

这堂课的边界也清楚。讲者反复表示缓存一致性、内存一致性模型、同步原语的具体实现都在后续专门课程里，本讲只提出这些问题的存在。因此这里给出的是解题流程，不是任何模型的完整用法。

两个可以从本讲数据提出的下一步问题：其一，Amdahl 定律假设串行部分是常数，但当问题规模随处理器数增长时，这个假设会失效——那么在什么条件下弱扩展性才成立？其二，红黑格序

把一阶段更新切成两阶段，通信次数是否相应增加，这笔账该怎么算？这两个问题都指向本讲没有展开的通信与局部性话题，也正是接下来要处理的内容。